HyperSpace AGI — Architettura di Distillazione Neurale CoT e Fine-Tuning dei Modelli Locali

Estensione Operativa delle Capacità di Chat Hook, Invocazione Skill e Strumenti Sovrani

Autore: Antigravity (HP OMEN 16 / Mesh Node) • **Destinatari:** Cips, Alberto, Team HyperSpace
Data: 21 Ottobre 2026 • **Ambiente:** HyperSpace v1.05 Mesh Cluster

1. Premessa e Visione Strategica

In questi mesi l'infrastruttura di HyperSpace si è arricchita di componenti sovrani di altissimo livello: il ponte neurale ComfyUI (SDXL Turbo, Pony, SD1.5, ControlNet), la memoria sovrana Hermes (SQLite WAL + FTS5), l'infrastruttura crittografica e di routing Mesh P2P (ECDSA inter-nodo), il Reel Editor automatico e il DAG orchestrator di Mission Control.

Tuttavia, è emerso un limite fondamentale: **i modelli linguistici compatti locali (SLM da 4B a 8B come Liquid AI LFM2.5 o Granite 3.3) sanno "tanto", ma nei chat hook operativi non sanno ancora "usare" concretamente questi strumenti**. Quando un utente richiede un'operazione via chat, il modello tende a rispondere con spiegazioni teoriche o ad allucinare invocazioni e parametri errati, mancando di un addestramento specifico sull'uso attivo delle nostre API.

Questo report definisce l'architettura della **Distillazione Neurale mediata da Tracce di Pensiero (Chain of Thought - CoT)** e il piano operativo per trasformare i nostri modelli in **agenti operativi capaci di attivare e orchestrare l'intero set di skill e tool di HyperSpace**.

2. Il Cuore Tecnologico: La Traccia di Pensiero (<think>)

Nei modelli compatti locali (4B–8B Q4_K_M residenti su schede consumer come la RTX 3070 8GB), l'inferenza diretta ("zero-shot") fallisce sistematicamente su compiti complessi per tre ragioni principali:

- **Allucinazione di API e Parametri:** il modello inventa metodi non esistenti o omette campi obbligatori.
- **Errori Sintattici e Refusi:** blocchi tronchi, mancata gestione di divisioni per zero o puntatori nulli.
- **Mancanza di Decomposizione:** incapacità di valutare sequenze causali prima di agire.

L'introduzione della **Traccia di Pensiero (<think>...</think>)** sdoppia l'inferenza in due fasi distinte:
1. **Fase di Ragionamento Privato (<think>):** uno spazio interiore in cui il modello analizza l'intento, mappa le dipendenze, controlla i vincoli di sistema (AGENTS.md) e pianifica il codice.
2. **Fase di Esecuzione Pubblica:** l'emissione rigorosa dell'output finale (codice Python verificabile, JSON schema strict o tool call normalizzata).

L'UniversalModelParser separa deterministicamente la traccia di pensiero dal payload finale, consentendo alle interfacce operative di consumare solo il risultato pulito.

3. Esempio Concreto dal Dataset Reale (HP OMEN 16)

Durante il turno notturno continuo, il worker autonomo omen_dev_loop_247.py ha distillato e validato **oltre 90 campioni sintetici completi** in data/synthetic_cot_dataset.jsonl. Di seguito è riportata una tupla reale generata da Liquid AI LFM2.5:

A. Prompt di Input (Compresso via Caveman Engine)

> *Crea una funzione Python `route_task_by_vram(task_requirements, nodes_list)` che calcola lo score di idoneità in base alla VRAM libera e all'alimentazione AC/Batteria. Mostra prima il ragionamento CoT in , poi restituisci il codice completo con type hinting.*

B. Traccia di Pensiero Reale Registrata (<think>)

```
The user wants a Python function `route_task_by_vram(task_requirements, nodes_list)`
that calculates a suitability score based on free VRAM and AC/Battery feeding.

1. Decomposizione dei requisiti:
   - task_requirements conterrà VRAM minima, eventuale soglia AC e batteria.
   - nodes_list è una lista di nodi mesh con attributi vram_available, ac_available, battery_available.
   - Punteggio finale normalizzato tra 0.0 e 1.0.

2. Risoluzione delle criticità ed edge cases:
   - Rischio ZeroDivisionError se req == 0: guardia `(avail / req) if req > 0 else 0.0`.
   - L'alimentazione AC è più stabile della batteria: pesi (VRAM 50%, AC 30%, Batteria 20%).
   - Se le risorse disponibili superano di molto il requisito, clamp: `max(0.0, min(1.0, score))`.

3. Pianificazione contratti e sintassi:
   - Importare Dict, List, Any da typing.
   - Docstring completa ed esecuzione autonoma.
```

C. Output Finale Validato via AST Python (ast_valid: true)

```python
from typing import Dict, List, Any

def route_task_by_vram(task_requirements: Dict[str, Any], nodes_list: List[Dict[str, Any]]) -> float:
    """Calcola lo score di idoneità (0.0-1.0) per VRAM e alimentazione."""
    required_vram = task_requirements.get("vram_needed", 0)
    required_ac = task_requirements.get("ac_needed", float('inf'))
    required_battery = task_requirements.get("battery_needed", float('inf'))

    vram_avail = [node.get("vram_available", 0) for node in nodes_list]
    ac_avail = [node.get("ac_available", 0.0) for node in nodes_list]
    battery_avail = [node.get("battery_available", 0.0) for node in nodes_list]

    # Prevenzione divisione per zero pianificata nel <think>
    vram_norm = [v / required_vram if required_vram > 0 else 0.0 for v in vram_avail]
    ac_norm = [a / required_ac if required_ac > 0 else 0.0 for a in ac_avail]
    battery_norm = [b / required_battery if required_battery > 0 else 0.0 for b in battery_avail]

    weights = {"vram": 0.5, "ac": 0.3, "battery": 0.2}
    score = (weights["vram"] * sum(vram_norm) +
             weights["ac"] * sum(ac_norm) +
             weights["battery"] * sum(battery_norm))

    return max(0.0, min(1.0, score))  # Clamp rigoroso definito nel reasoning
```

4. Applicazione Architetturale nei Componenti di HyperSpace

Questa metodologia è integrata in modo trasversale attraverso l'architettura di HyperSpace:

1. **Distillazione & Fine-Tuning a Costo Zero:** il worker notturno genera centinaia di tuple convalidate da usare come dataset supervisionato per training QLoRA locale su PyTorch.
2. **Pianificazione DAG in Mission Control:** il planner decompone con think=True in passaggi discreti, mentre i worker eseguono senza latenza aggiuntiva con /no_think.
3. **Pipeline Pre/Post-Inference (Universal Parser):** il CavemanEngine comprime i prompt del 20-40% preservando contratti e tag, mentre il parser ripara JSON parziali da troncamento token.
4. **Self-Healing Loop:** se un test fallisce, il messaggio del compilatore viene re-iniettato nel prompt per l'auto-riparazione immediata guidata da CoT.

5. **Memoria Sovrana Hermes:** le lezioni apprese dai blocchi CoT vengono salvate nel database SQLite FTS5 per orientare le sessioni future.

5. La Nuova Frontiera: Dal Sapere al Saper Fare (Tool-Use nei Chat Hook)

La priorità strategica evidenziata è **insegnare ai modelli a usare attivamente tools e skill nei chat hook**. Attualmente i modelli compatti tendono a produrre risposte discorsive invece di invocare le azioni reali.

Il Nuovo Schema di Ragionamento CoT per Tool-Calling:
Nei chat hook (Telegram, Discord, WebUI Cockpit), il modello viene addestrato a compiere una sequenza deterministica a 4 stadi:

- **Rilevamento Intento:** discernere se l'utente desidera una conversazione o un'azione operativa sul cluster.
- **Selezione del Tool:** individuare il tool registrato idoneo (es. hermes_memory_store, comfy_generate, reel_render).
- **Validazione Parametri & Sicurezza:** verificare tipi, percorsi e assenza di credenziali esposte (policy AGENTS.md).
- **Emissione Strutturata:** emettere rigorosamente il payload <tool_call>{...}</tool_call> senza testo superfluo.

Esempio di Traccia CoT & Chiamata Tool Strutturata per Chat Hook

```
PROMPT: "Registra nella memoria che su Windows i file SQLite richiedono conn.close() prima del cleanup."

<think>
1. Intento: Salvataggio permanente di una lezione appresa sul repository.
2. Tool: Hermes Memory Bridge (hermes_memory_store).
3. Parametri: text esplicito con WinError 32, category "lesson_learned", tags ["windows", "sqlite", "resilience"].
4. Output: emetto solo il payload tool_call conforme allo schema.
</think>

<tool_call>
{
  "tool": "hermes_memory_store",
  "arguments": {
    "text": "Su host Windows, le connessioni SQLite devono essere chiuse esplicitamente prima della cancellazione
    "category": "lesson_learned",
    "tags": ["windows", "sqlite", "resilience"]
  }
}
</tool_call>
```

6. Piano Operativo e Prossimi Passi Condivisi

1. **Espansione DEV_TOPICS:** estendere il generatore omen_dev_loop_247.py introducendo i 4 domini di tool-use (ComfyUI, Hermes Memory, Reel Editor, Mesh Telemetry).
2. **Validatore di Schema Tool Call:** aggiungere un quality gate che validi la conformità JSON dei parametri contro i manifest di HyperSpace prima di ammettere il campione nel dataset.
3. **Fine-Tuning QLoRA:** al raggiungimento di 500 campioni convalidati, avviare il training notturno su GPU RTX 3070 ed esportare gli adapter per Ollama.
4. **Attivazione Chat Hook:** agganciare il modello sintonizzato sui bot Telegram/Discord con esecuzione automatica e trasparente delle skill.